



Architecture Infrastructure as Code (Terraform)

Cette documentation détaille les choix techniques et la structure des fichiers Terraform utilisés pour le déploiement multi-cloud de **GeoMeteo**.



Pourquoi avoir choisi Terraform ?

Plutôt que d'utiliser les consoles web des fournisseurs ou des solutions propriétaires (comme Render Blueprint ou AWS CloudFormation), nous utilisons **Terraform** pour trois raisons stratégiques :

1. **Agnosticité Cloud (Anti Vendor Lock-in)** : L'application ne dépend d'aucun fournisseur spécifique. Si les tarifs de Render augmentent ou si AWS subit une panne majeure, nous pouvons migrer l'intégralité du service vers GCP ou Azure en changeant simplement de dossier.
2. **Reproductibilité** : L'infrastructure est décrite par le code. Chaque déploiement est identique, éliminant les erreurs humaines de configuration manuelle.
3. **Documentation Vivante** : Le code Terraform sert de documentation officielle pour l'architecture réseau et serveur.



Anatomie des fichiers de configuration

Chaque module de fournisseur (AWS, GCP, Azure, etc.) repose sur une structure standardisée en trois fichiers pour garantir une maintenance simplifiée.

1. variables.tf : L'interface d'entrée

Ce fichier définit les paramètres requis pour le déploiement.

- **Rôle** : Il agit comme un "contrat". Il liste ce que l'infrastructure a besoin de savoir (ex: api_key, image_url) sans pour autant fixer les valeurs.
- **Justification** : En isolant les variables, on permet au code d'être réutilisable. Un développeur peut déployer sa propre instance sans toucher à la logique du serveur.

2. main.tf : Le cœur de l'orchestration

C'est ici que sont définies les ressources cloud réelles.

- **Rôle** : Il contient la déclaration des services (ex: google_cloud_run_v2_service pour GCP ou aws_apprunner_service pour AWS).
- **Justification** : Nous utilisons principalement des services de type **CaaS (Container as a Service)**. Cela nous permet de bénéficier de la puissance du Cloud (auto-scaling, HTTPS natif) tout en gardant une configuration logicielle minimale grâce à Docker.

3. outputs.tf : Le retour d'information

Une fois le déploiement terminé, Terraform ne nous laisse pas "deviner" où se trouve l'application.

- **Rôle** : Il extrait les données générées par le Cloud (comme l'URL générée dynamiquement) et les affiche dans la console.
- **Justification** : Indispensable pour l'expérience développeur (DX) et pour l'intégration avec des pipelines CI/CD qui auraient besoin de récupérer l'URL pour lancer des tests de fumée (Smoke Tests).



Gestion des Secrets & Sécurité

La sécurité est au centre de notre démarche IaC :

- **Exclusion des Secrets** : Le fichier terraform.tfvars (contenant les vraies clés) et les fichiers .tfstate (contenant l'état de l'infrastructure) sont strictement ignorés par Git via le .gitignore.
- **Injection Dynamique** : Pour les environnements de production, nous privilégions l'injection via des variables d'environnement sécurisées dans GitHub Actions plutôt que le stockage de fichiers sur disque.



Stratégie Multi-Cloud

Le projet démontre trois types d'architectures gérées par Terraform :

1. **Serverless Containers** : Idéal pour la mise sur le marché rapide (GCP, Azure, Scaleway).
2. **IaaS avec Cloud-Init** : Pour un contrôle total des coûts sur des serveurs nus (Hetzner, OVH).
3. **Enterprise Networking** : Pour les environnements complexes nécessitant des VPC et des Security Groups dédiés (Oracle, Alibaba).

Document conçu pour accompagner le déploiement de GeoMeteo v1.0.